Функции

функции, формальные параметры, локальные переменные, глобальные переменные, ключевые слова void и return

Проблема дублирования кода

```cpp
#include <iostream>

using namespace std;

int main() {
    unsigned long long a, b, a_sum, b_sum;
    cin >> a >> b;

    //process 'a' varialbe
    {
        auto sum = 0ull, tmp = a;
        do
            sum += tmp%10;
        while (tmp /= 10);
        a_sum = sum;
    }

    //prodess 'b' variable
    {
        auto sum = 0ull, tmp = b;
        do
            sum += tmp%10;
        while (tmp /= 10);
        b_sum = sum;
    }

    cout << a_sum << " " << b_sum << endl;

    return 0;
}
```

Эти переменные имеют смысл только для цикла и заключены в блок кода

Для переменной b код повторяется

Дублированием кода называется необязательное повторение одинаковых фрагментов кода в различных частях программы.

Дублированием кода может приводить к проблемам при обслуживании кода (maintenance): при внесении изменений возникнет необходимость поиска и замены одинаковых фрагментов по всему объёму кода, например при отладке.

Функции

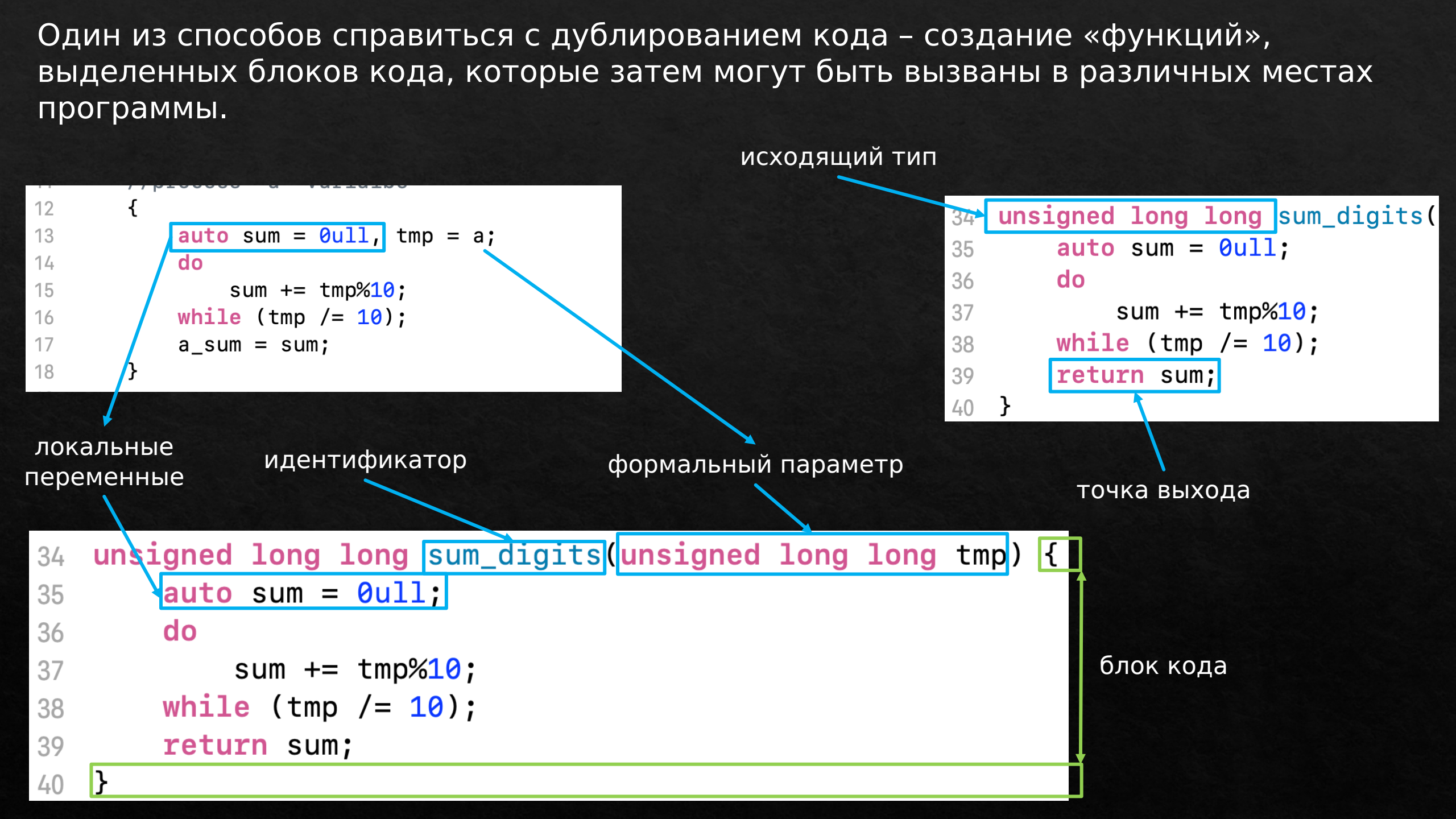
функции как поименованные блоки кода

Один из способов справиться с дублированием кода – создание «функций», выделенных блоков кода, которые затем могут быть вызваны в различных местах программы.

```
12      {
13          auto sum = 0ull, tmp = a;
14          do
15              sum += tmp%10;
16          while (tmp /= 10);
17          a_sum = sum;
18      }
```

```
34  unsigned long long sum_digits(
35          auto sum = 0ull;
36          do
37              sum += tmp%10;
38          while (tmp /= 10);
39          return sum;
40  }
```

```
34  unsigned long long sum_digits(unsigned long long tmp) {
35          auto sum = 0ull;
36          do
37              sum += tmp%10;
38          while (tmp /= 10);
39          return sum;
40  }
```



Использование функции

```
34 unsigned long long sum_digits(unsigned long long tmp) {
35     auto sum = 0ull;
36     do
37         sum += tmp%10;
38     while (tmp /= 10);
39     return sum;
40 }
41
42 int main() {
43     unsigned long long a, b, a_sum, b_sum;
44     cin >> a >> b;
45
46     //process 'a' varialbe
47     a_sum = sum_digits(a);
48
49     //prodess 'b' variable
50     b_sum = sum_digits(b);
51
52     cout << a_sum << " " << b_sum << endl;
53
54     return 0;
55 }
```

описание функции

аргумент функции

вызов функции

результат работы функции мы сохраняем в переменную

Использование функции

Сохранение результата работы функции в промежуточную переменную не является обязательным. Вызов функции может быть часть более сложного выражения.

```
43  int main() {
44      unsigned long long a, b;
45      cin >> a >> b;
46
47      cout << sum_digits(a) << " " << sum_digits(b) << endl;
48
49      auto full_sum = sum_digits(a) + sum_digits(b);
50
51      return 0;
52  }
```

Использование функции

```
34 unsigned long long sum_digits(unsigned long long number) {
35     auto sum = 0ull;
36     do
37         sum += number%10;
38     while (number /= 10);
39     return sum;
40 }
41
42 unsigned long long digit_reduce(unsigned long long number) {
43     while(number > 9ull)
44         number = sum_digits(number);
45     return number;
46 }
47
48 int main() {
49     unsigned long long number;
50     cin >> number;
51     cout << digit_reduce(number) << endl;
52
53     return 0;
54 }
```

функцию можно вызвать из другой функции

Пример

```cpp
#include <iostream>

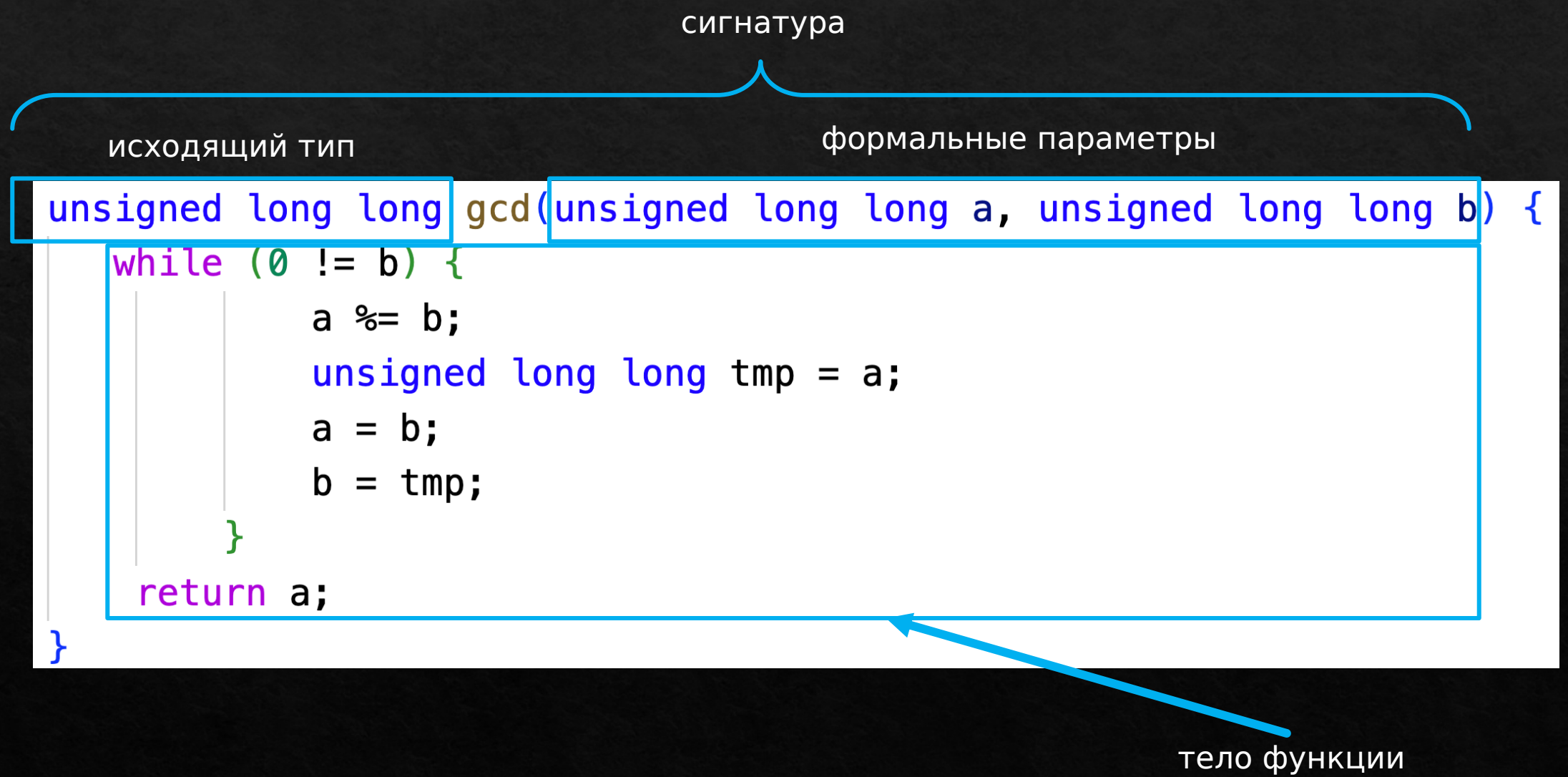
int main() {
    unsigned long long a, b, gcd;
    std::cin >> a >> b;
    {
        while (0 != b) {
            a %= b;
            unsigned long long tmp = a;
            a = b;
            b = tmp;
        }
        gcd = a;
    }
    std::cout << gcd << std::endl;

    return 0;
}
```

```cpp
unsigned long long gcd(
    unsigned long long a,
    unsigned long long b
) {
    while (0 != b) {
            a %= b;
            unsigned long long tmp = a;
            a = b;
            b = tmp;
        }
    return a;
}
```

```cpp
int main() {
    unsigned long long a, b;
    std::cin >> a >> b;
    std::cout << gcd(a, b) << std::endl;
}
```

Немного умных слов



- Функция может принимать произвольное количество параметров, в том числе ни одного
- Функция может возвращать только одно значение либо не возвращать ничего
- Функцию можно вызвать в коде неограниченное число раз, в том числе из тела другой функции
- Вызов функции сам по себе может быть частью более сложного выражения

Локальные и глобальные переменные

функция не возвращает значения

```
7  void swap_us(unsigned long long a, unsigned long long b) {
8      cout << "In function before swap: " << a << " " << b << '\n';
9      auto tmp = a;
10     a = b;
11     b = tmp;
12     cout << "In function after swap: " << a << " " << b << '\n';
13 }
14
15 int main() {
16     auto a = 5ull, b = 23ull;
17     cout << "In main before swap: " << a << " " << b << '\n';
18     swap_us(a,b);
19     cout << "In main after swap: " << a << " " << b << '\n';
20 }
```

```
In main before swap: 5 23
In function before swap: 5 23
In function after swap: 23 5
In main after swap: 5 23
```

глобальные переменные

```cpp
unsigned long long a, b;

void swap_us() {
    cout << "In function before swap: " << a << " " << b << '\n';
    auto tmp = a;
    a = b;
    b = tmp;
    cout << "In function after swap: " << a << " " << b << '\n';
}

int main() {
    a = 5ull; b = 23ull;
    cout << "In main before swap: " << a << " " << b << '\n';
    swap_us();
    cout << "In main after swap: " << a << " " << b << '\n';
}
```

```
In main before swap: 5 23
In function before swap: 5 23
In function after swap: 23 5
In main after swap: 23 5
```

Локальные переменные видны только в рамках блока кода в котором объявлены, например в теле функции. Локальные переменные могут быть изменены только в той функции, где объявлены, что защищает их от случайного изменения.

Глобальные переменные видны в любом месте программы. Глобальные переменные могут быть непредсказуемо изменены в любой части кода.

Использование глобальных переменных может привести к сложной связи между разными функциями или даже к скрытым зависимостям в коде.

```
7  unsigned long long i;
8
9  unsigned long long increment() {
10     return ++i;
11 }
12
13 void reset() {
14     i = 0;
15 }
16
17 int main() {
18     reset();
19     while(i < 10)
20         increment();
21
22     i = 15;
23     while(i < 25)
24         increment();
25 }
```

Две функции и функция main связаны между собой через глобальную переменную.

Значения по умолчанию

Если функция используется с определённым значением параметра часто

```
unsigned next(unsigned const input, unsigned const step) {
    return input + step;
}
```

Функция next в коде часто может использоваться с определённым параметром шага

```
int main() {
    unsigned a = 0;
    for (unsigned counter = 0; counter != 10; ++counter)
        std::cout << next(a, 1u) << std::endl;
    return 0;
}
```

В этой ситуации наиболее очевидным решением будет создать специальную функцию, которая вызывает исходную функцию, задавая один из параметров конкретным числом.

```
unsigned next_one(unsigned const input) {
    return next(input, 1u);
}
```

Определение значения по умолчанию

```
unsigned next(unsigned const input, unsigned const step) {
    return input + step;
}
```

Указываем у соответствующего формального параметра значение, которое будет использоваться, если аргумент опущен

```
unsigned next(unsigned const input, unsigned const step = 1u) {
    return input + step;
}
```

```
int main() {
    unsigned a = 0;
    std::cout << next(a, 1u) << std::endl;
    std::cout << next(a)     << std::endl;
    std::cout << next_one(a) << std::endl;
    return 0;
}
```



Определение значения по умолчанию

```
int minimax(int a, int min, int max) {
    return std::min(max, std::max(min, a));
}
```

Функция позволяет гарантировать, что результат вызова будет не меньше min и не больше max

```
int grad(int g) {
    return minimax(g, 0, 10);
}
```

Мы часто используем эту функцию с промежутком от 0 до 10

Определим два значения по умолчанию:

```
int minimax(int a, int min = 0, int max = 10) {
    return std::min(max, std::max(min, a));
}
```

```
std::cout << minimax(-1) << std::endl;
std::cout << minimax(5)  << std::endl;
std::cout << minimax(11) << std::endl;
```

```
0
5
10
```

- Функция может иметь любое количество значений по умолчанию, но обязательно должны быть определены «с хвоста»

```
int minimax(int a, int min = 0, int max) {
    return std::min(max, std::max(min, a));
}
```

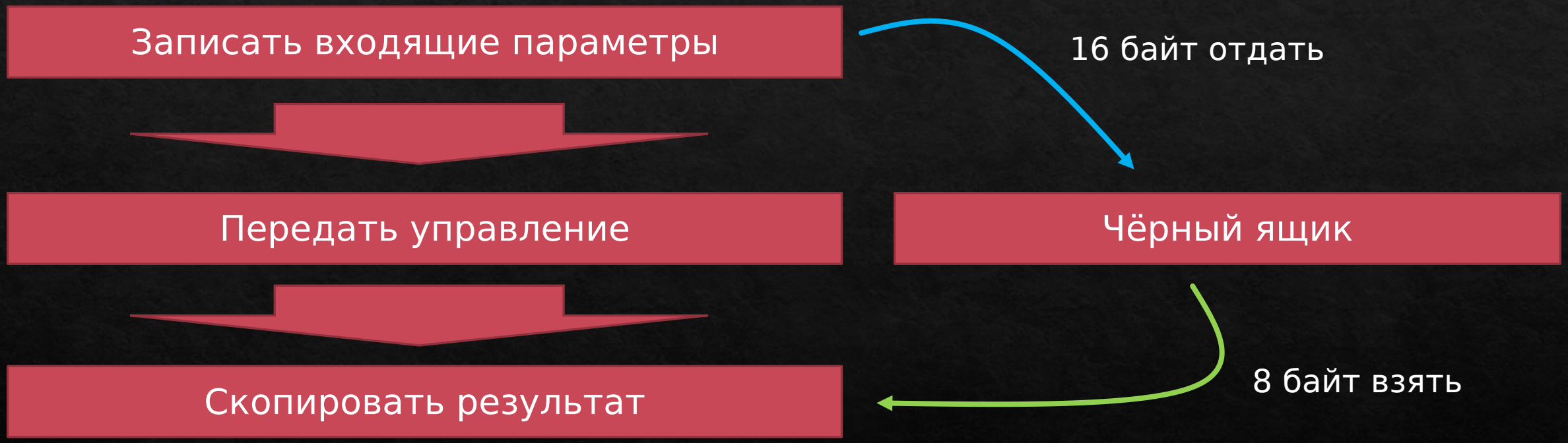
- При вызове функции аргументы подставляются слева направо. Нельзя подставить последнее значение в некоторый параметр, оставив левый значением по умолчанию
- Значениями по умолчанию могут быть не только литеральные константы, но они должны быть известны на этапе компиляции

```
int minimax(
    int a,
    int min = std::numeric_limits<int>::min(),
    int max = std::numeric_limits<int>::max()
) { return std::min(max, std::max(min, a)); }
```

Предварительная декларация

Что нужно, чтобы вызвать функцию?

```
unsigned long long gcd(unsigned long long a, unsigned long long b)
```



Сам код функции мы можем рассматривать как чёрный ящик. Как конкретно функция исполняется мы можем не задумываться. Главное – удовлетворить ограничениям входящих параметров и получить результат.

Мы можем сначала описать как вызвать функцию, а как именно она работает описать позже.

```
unsigned long long gcd(unsigned long long a, unsigned long long b);

int main() {
    unsigned long long a, b;
    std::cin >> a >> b;
    std::cout << gcd(a, b) << std::endl;
}

unsigned long long gcd(unsigned long long a, unsigned long long b) {
   while (0 != b) {
            a %= b;
            unsigned long long tmp = a;
            a = b;
            b = tmp;
        }
    return a;
}
```

предварительная декларация

Значения по умолчанию следует указывать только в объявлении функции

```
48  int minimax(
49      int a,
50      int min = std::numeric_limits<int>::min(),
51      int max = std::numeric_limits<int>::max()
52  );
53
54  int main() {
55      std::cout << minimax(-1) << std::endl;
56      std::cout << minimax(5)  << std::endl;
57      std::cout << minimax(11) << std::endl;
58      return 0;
59  }
60
61  int minimax(int a, int min, int max) {
62      return std::min(max, std::max(min, a));
63  }
```

```
-1
5
11
```

Перегрузка функций

Общее представление о перегрузке функций

Перегруженными называются функции, которые имеют одинаковые идентификаторы, но различаются формальными параметрами – типами или количеством.

```
int func(int a);
int func(int a, int b);
float func(float a);
float func(float a, float b);
float func(float a, int b);
```

Пример перегрузок (overload) функции func.

Перегруженные функции используются для описания алгоритмов, которые могут различаться в реализации для различных типов данных или количества входящих данных, либо для более оптимальной работы функций с одним или несколькими заданными аргументами.

Общее представление о перегрузке функций

```
int max3(int a, int b, int c) {
    return std::max(a, std::max(b, c));
}

bool max3(bool a, bool b, bool c) {
    return a || b || c;
}
```

Максимальное значение из трёх элементов типа bool можно найти несколько иным алгоритмом.

```
int minimax(int a, int min, int max) {
    return std::min(max, std::max(min, a));
}

int minimax(int a, int min) {
    return std::max(min, a);
}
```

Перегрузка функций как вариант алгоритма с одним из параметров заданным по умолчанию.

Встраиваемые функции

Функция может быть настолько короткой, что время вызова этой функции окажется сравнимо с выполнением самой функции. Такие функции можно «встраивать» – вместо вызова функции в точке использования компилятор подставляет непосредственно тело функции.

```
int my_max(int a, int b) {
    return a > b ? a : b;
}
```

```
int main() {
    unsigned x = 3, y = 5;
    std::cout << my_max(x, y) << std::endl;

    return 0;
}
```

```
int main() {
    unsigned x = 3, y = 5;
    std::cout << (x > y ? x : y) << std::endl;

    return 0;
}
```

Чтобы подсказать компилятору, что некоторая функция может быть встроена, желательно, чтобы была встроена, необходимо указать ключевое слово inline перед определением функции.

```
inline
int my_max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    unsigned x = 3, y = 5;
    std::cout << my_max(x, y) << std::endl;

    return 0;
}
```

Следует помнить, что функция не обязательно будет встроена, компилятор обладает большей информацией о конечном коде, чтобы вычислить скорость вызова и скорость работы функции. Равно не следует полагаться, что такая функция вообще будет существовать в виде функции, возможно все её вызовы будут встроены и создание кода функции вовсе не потребуется.

Многие компиляторы позволяют дополнительными средствами определить, что некоторая функция всегда должна быть встроена.